Université Abdelmalek Essaâdi

Faculté des Sciences et Techniques de Tanger
Département Génie Informatique
Master MBD

Atelier 1
Deep Learning avec PyTorch

Régression boursière et classification
multiclasse des pannes

Réalisé par **Karmas Hamam**
Enseignant : Pr. Lotfi Elaachak

Rapport détaillé : données, méthode, implémentation,
résultats, interprétation et reproduction

Les résultats présentés proviennent des expériences complètes enregistrées dans le dépôt. Aucun résultat supplémentaire n'est supposé.

Résumé du travail

L'objectif de cet atelier est de comprendre le fonctionnement d'un réseau de neurones multicouche avec PyTorch, puis de l'appliquer à deux problèmes différents. La première partie prédit une valeur continue : le prochain cours de clôture d'Apple. La seconde prédit une catégorie : le type de panne d'une machine.

Le travail suit une même logique : comprendre les données, définir la cible, séparer les observations, préparer les entrées, rechercher des hyperparamètres, comparer plusieurs régularisations et évaluer le modèle sur des données conservées à part.

Élément	Régression	Classification
Données	NYSE : prix ajustés, action AAPL	Maintenance prédictive : 10 000 observations
Objectif	Clôture de la prochaine séance	Six classes de Failure Type
Score de sélection	RMSE minimale	F1 macro maximal
Modèle choisi sur validation	combined	dropout
Résultat test principal	RMSE = 1.4919 USD	F1 macro = 0.5668
Référence simple	Persistance : RMSE = 1.5065	Classe majoritaire : F1 macro = 0.1637

Ce que les résultats permettent de conclure

En régression, le réseau sélectionné est très proche d'une simple persistance du dernier prix. Le résultat ne démontre pas une capacité de prévision boursière forte. En classification, le F1 macro progresse par rapport à la prédiction systématique de la classe majoritaire, mais certaines pannes rares restent mal détectées. Le modèle ne constitue donc pas un système industriel validé.

La valeur de ce travail est aussi méthodologique : les prétraitements sont ajustés dans chaque fold, le test ne guide pas les choix et les résultats peu favorables sont conservés et expliqués.

Sommaire

Résumé du travail 2

1. Comprendre le sujet et les livrables 4

2. Méthode générale et prévention des fuites 5

3. Régression : données, cible et variables 6

4. Régression : exploration et préparation 7

5. Régression : réseau et hyperparamètres 8

6. Régression : résultats et interprétation 9

7. Classification : comprendre les données 11

8. Classification : EDA et rééquilibrage 12

9. Classification : MLP et recherche 14

10. Classification : résultats globaux 15

11. Classification : métriques par classe 17

12. Matrice de confusion et diagnostic 18

13. Organisation du code et reproduction 19

14. Synthèse personnelle et conclusion 20

Annexe A — Détail de la recherche CV 21

Annexe B — Lire les points clés du code 22

Annexe C — Sources, traçabilité et glossaire 23

1. Comprendre le sujet et les livrables

1.1 Les objectifs pédagogiques

Le sujet demande de se familiariser avec PyTorch pour construire des architectures DNN/MLP. Un MLP est un réseau constitué de couches entièrement connectées : chaque neurone reçoit les sorties de la couche précédente. Les couches cachées apprennent des relations non linéaires entre les variables.

La régression et la classification partagent la préparation des données et la boucle d'apprentissage. Elles diffèrent surtout par la cible, la dernière couche, la fonction de perte et les métriques d'évaluation.

Question du sujet	Réalisation dans le projet
Exploration et nettoyage	Statistiques, valeurs manquantes, doublons, distributions et corrélations
DNN avec PyTorch	Module MLP, ReLU et boucle d’apprentissage explicite
Recherche des hyperparamètres	GridSearchCV : neuf configurations, trois folds par tâche
Courbes par époque	Loss et RMSE ou accuracy sur train, validation et test
Régularisation	Dropout, L2, arrêt anticipé et combinaison
Déséquilibre des classes	SMOTENC dans le pipeline de classification
Évaluation et interprétation	Références simples, scores globaux et par classe
Remise	Notebooks, code, README, synthèse et présent rapport PDF

1.2 Deux précisions nécessaires

Le sujet ne fixe pas la variable à prédire pour NYSE. Le choix retenu est donc explicite : à la clôture du jour t, prévoir la clôture ajustée de la prochaine séance pour AAPL. Le sujet demande aussi une accuracy en régression ; cette métrique étant inadaptée à une valeur continue, elle est remplacée par la RMSE.

Pour la partie multiclasse, la cible choisie est Failure Type. La colonne Target, qui indique seulement panne ou absence de panne, ne répondrait pas à la demande multiclasse et révélerait une information liée à la cible si elle était utilisée comme entrée.

2. Méthode générale et prévention des fuites

2.1 Le rôle des trois partitions

Entraînement : apprendre les transformations et les poids du réseau. La validation croisée y compare les hyperparamètres. **Validation indépendante :** comparer les régularisations et surveiller l'arrêt anticipé. **Test :** mesurer les performances finales après avoir fixé tous les choix.

Une fuite de données apparaît lorsqu'une information indisponible au moment de la prédiction ou provenant de la validation/test influence l'apprentissage. Cela produit des scores trop optimistes. Ici, imputation, standardisation et rééquilibrage sont intégrés dans les pipelines afin d'être ajustés seulement sur le train de chaque fold.

Étape	Règle appliquée
Imputation et standardisation	Apprendre les paramètres sur le train du fold, transformer les autres partitions sans les réajuster
Standardisation de la cible de régression	Moyenne et écart-type calculés dans fit sur la cible du train
Augmentation SMOTENC	Uniquement dans les folds train ; aucune augmentation de validation/test
Sélection des variantes	RMSE de validation en régression, F1 macro de validation en classification
Arrêt anticipé	Loss de validation, patience de dix époques et restauration des meilleurs poids
Courbes de test	Calcul rétrospectif à partir des poids sauvegardés, après sélection

2.2 Lire correctement la validation croisée

Chaque tâche compare neuf configurations sur trois folds, soit 27 ajustements. Au total, les deux recherches effectuent 54 ajustements, auxquels s'ajoutent dix modèles finaux. Les colonnes mean_test_score des fichiers GridSearchCV désignent le score sur les folds de validation croisée, et non sur le test final du projet.

2.3 Reproductibilité

La graine est fixée à 42 pour Python, NumPy et PyTorch. Le réseau utilise le CPU, deux threads PyTorch et des algorithmes déterministes. Les versions principales enregistrées sont Python 3.12.14, PyTorch 2.14.1+cpu et scikit-learn 1.8.0. Une autre plateforme peut encore produire de petites différences numériques.

3. Régression : données, cible et variables

Le fichier prices-split-adjusted.csv contient 851,264 lignes et sept colonnes, pour plusieurs actions. Les prix ajustés réduisent les ruptures liées aux divisions d'actions. Pour éviter de mélanger les niveaux de prix de sociétés différentes, cette étude se limite à Apple, symbole AAPL.

Après tri chronologique et création des variables, 1 757 observations sont exploitables. Les premières lignes sans cinq séances d'historique et la dernière ligne sans clôture future sont retirées. Le contrôle du fichier brut ne détecte ni valeur manquante ni doublon exact.

Variable	Signification et disponibilité
open, close, low, high	Prix d’ouverture, clôture, minimum et maximum de la séance t, connus à sa clôture
volume	Volume échangé pendant la séance t
return_1	Rendement close(t) / close(t-1) - 1
mean_5	Moyenne des cinq dernières clôtures, y compris la séance t
std_5	Écart-type des cinq dernières clôtures
next_close — cible	Clôture de la prochaine séance, créée par shift(-1) ; absente des entrées
target_date	Date associée à la cible, utilisée pour contrôler les frontières

3.1 Séparation chronologique avec purge

Partition	Lignes	Dates des entrées	Dernière cible
Train	1 228	08/01/2010 – 21/11/2014	24/11/2014
Validation	263	25/11/2014 – 10/12/2015	11/12/2015
Test	264	14/12/2015 – 29/12/2016	30/12/2016

Le partage vise environ 70 % / 15 % / 15 %. Une ligne est purgée à chaque frontière afin que la date cible d'une partition soit strictement antérieure à la première date d'entrée de la suivante. La validation croisée utilise aussi TimeSeriesSplit(n_splits=3, gap=1). Un partage aléatoire serait inadapté à cette expérience temporelle.

4. Régression : exploration et préparation

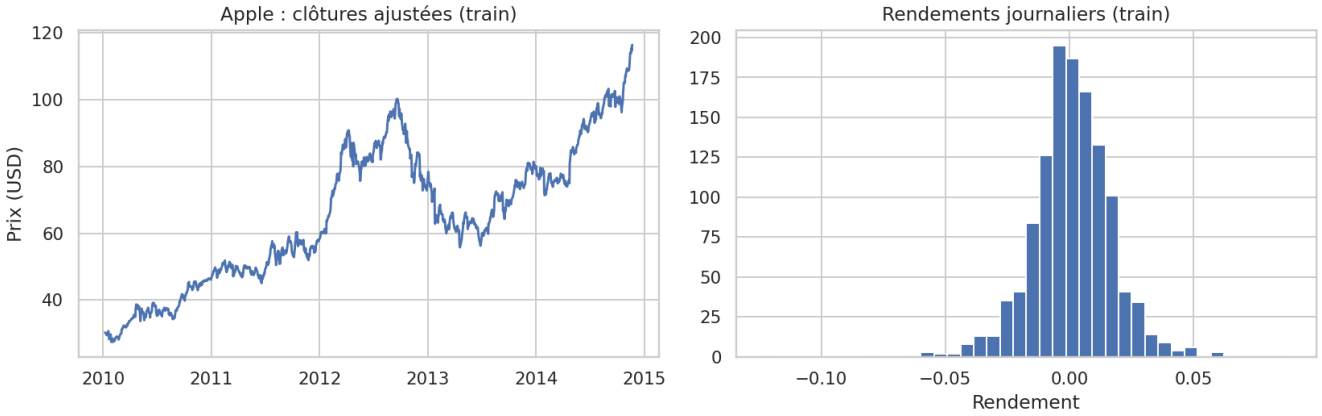


Figure 1 — Évolution du cours et distribution des rendements, uniquement sur le train.

Les cours évoluent fortement sur la période d'entraînement : leur niveau n'est pas constant dans le temps. Les rendements sont davantage concentrés autour de zéro, avec quelques variations plus marquées. Cette différence explique pourquoi un réseau qui apprend le niveau du prix peut afficher un R^2 élevé sans prédire précisément ses variations.

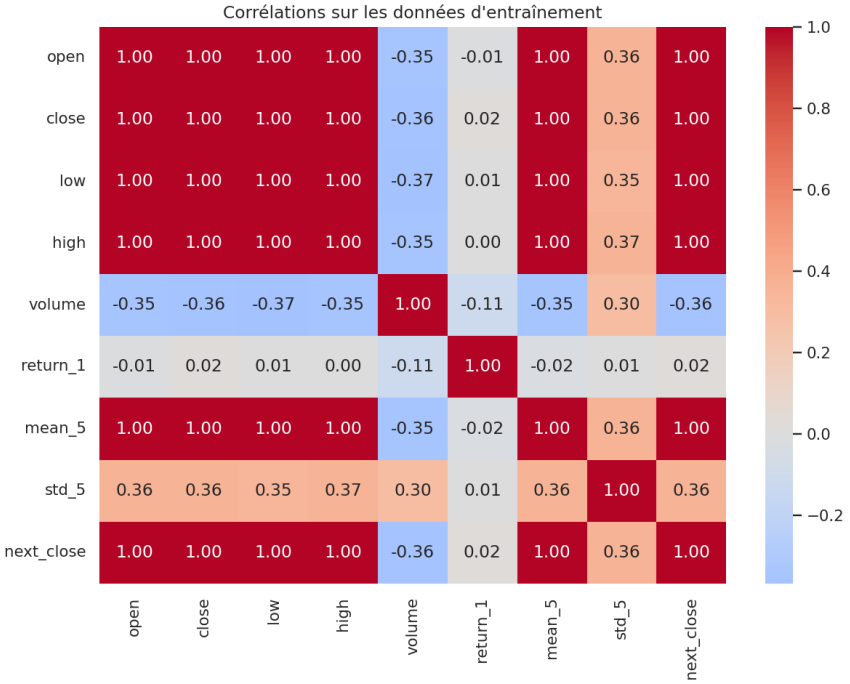


Figure 2 — Corrélations entre les entrées et la cible sur le train.

Les prix de la même séance et les moyennes récentes sont fortement liés. La clôture de la prochaine séance est également proche des clôtures récentes. Une corrélation forte n'est pas une preuve de causalité et justifie une comparaison à la persistance. Les variables sont standardisées pour éviter que le volume, beaucoup plus grand en échelle, domine numériquement les prix.

5. Régression : réseau et hyperparamètres

5.1 Architecture et fonction de perte

Le modèle retenu par validation croisée comporte huit entrées, une couche cachée de 32 neurones avec activation ReLU et une sortie linéaire. Il contient 321 paramètres entraînables : 8 × 32 + 32, puis 32 × 1 + 1. La variante combinée ajoute un Dropout de 0,2 après l'activation, sans ajouter de paramètres entraînables.

```
8 variables → Linear(8, 32) → ReLU → [Dropout(0.2)]
            → Linear(32, 1) → prix prédit
```

La cible est standardisée pendant fit : y_standard = (y - moyenne_train) / écart_type_train. Le réseau minimise la MSE sur cette cible. Lors de predict, la transformation inverse remet les prédictions en dollars. Les courbes de loss présentées sont recalculées en MSE exprimée en USD² pour être interprétables.

5.2 Grille effectivement exécutée

La grille combine Adam avec les architectures [32] ou [64, 32], des taux d'apprentissage 0,001 ou 0,01 et 40 ou 80 époques : huit configurations. Une neuvième configuration teste SGD avec momentum 0,9, architecture [64, 32], taux 0,01 et 80 époques. La taille des mini-lots vaut 512 ; elle n'est pas optimisée.

Pour relier PyTorch à scikit-learn, TorchRegressor et TorchClassifier suivent l'interface BaseEstimator : fit apprend les poids, predict calcule les prédictions et les paramètres du constructeur peuvent être clonés. GridSearchCV crée ainsi un modèle neuf pour chaque configuration et chaque fold, sans réutiliser les poids d'un autre ajustement.

La meilleure configuration utilise Adam, le taux 0,01, [32] et 80 époques. La RMSE moyenne CV vaut 1.3680 USD. Scikit-learn enregistre son opposé pour maximiser le score : plus la RMSE négative est proche de zéro, meilleur est le résultat.

5.3 Comment les poids sont appris

```
pour chaque époque :
    pour chaque mini-lot du train :
        optimizer.zero_grad()
        prediction = model(entrees)
        loss = criterion(prediction, cible)
        loss.backward()
        optimizer.step()
    mesurer train et validation sans gradient
```

Une époque correspond à un passage sur tout le train. Le taux d'apprentissage contrôle l'amplitude des mises à jour ; l'optimiseur détermine leur calcul. Un nombre trop faible d'époques peut laisser le modèle sous-entraîné, tandis qu'un apprentissage trop long peut favoriser le surapprentissage.

6. Régression : résultats et interprétation

Variante	RMSE validation	MAE test	RMSE test	R² test
initial	2.0718	1.0495	1.4661	0.9617
dropout	2.9417	1.7062	2.0552	0.9248
l2	2.0294	1.0584	1.4843	0.9608
early_stopping	2.0530	1.1028	1.5143	0.9592
combined	2.0018	1.0533	1.4919	0.9604

Le choix final est **combined**, car cette variante présente la plus faible RMSE sur validation parmi les cinq réseaux comparés. Le modèle initial a une RMSE test légèrement meilleure, mais le choisir après lecture du test rendrait l'évaluation biaisée. L'ordre des modèles sur validation n'est pas garanti sur une autre période.

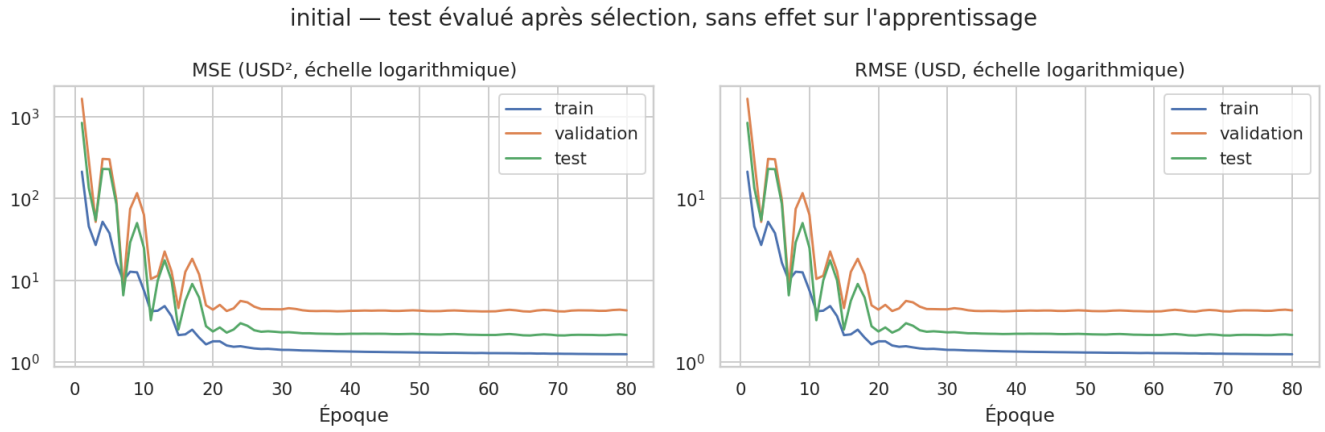


Figure 3 — Modèle initial : MSE et RMSE par époque, échelles logarithmiques.

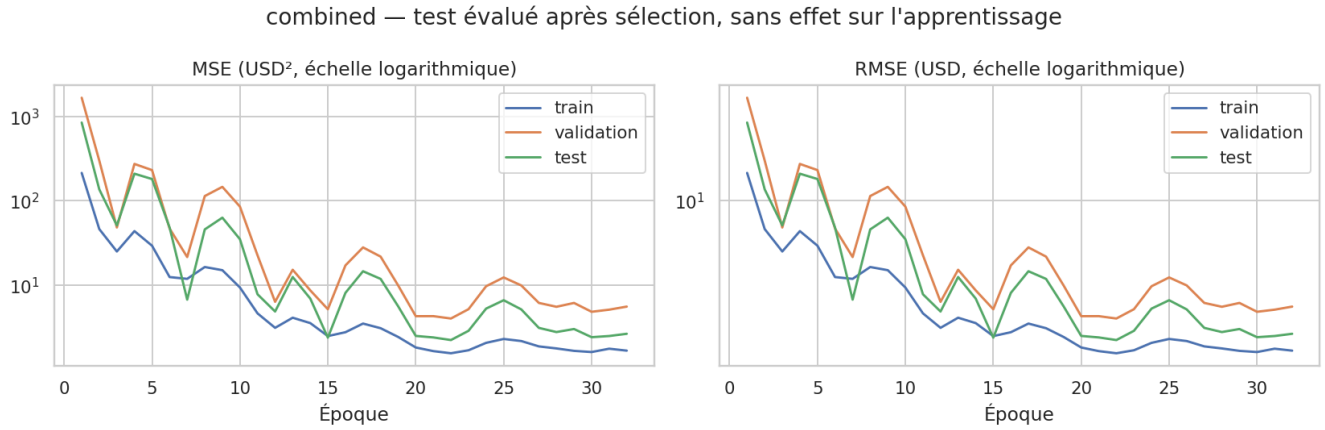


Figure 4 — Variante combinée : entraînement arrêté après 32 époques, poids de l'époque 22 restaurés.

Les erreurs baissent rapidement, puis se stabilisent. L'erreur de validation reste supérieure à celle du train, ce qui invite à surveiller la généralisation. Le Dropout seul dégrade ici la validation : la régularisation réduit la capacité effective du réseau et ne garantit pas une amélioration.

6.1 Comparer à une référence exigeante

Méthode	RMSE validation	MAE test	RMSE test	R² test
Persistance	1.9592	1.0648	1.5065	0.9596
MLP choisi	2.0018	1.0533	1.4919	0.9604

La persistance prédit simplement : clôture prochaine = clôture actuelle. Elle obtient une meilleure RMSE sur validation que le réseau choisi. Sur le test, le gain du MLP reste minime et sa significativité n'a pas été évaluée. Les résultats ne démontrent ni un avantage économique ni une stratégie de trading rentable.

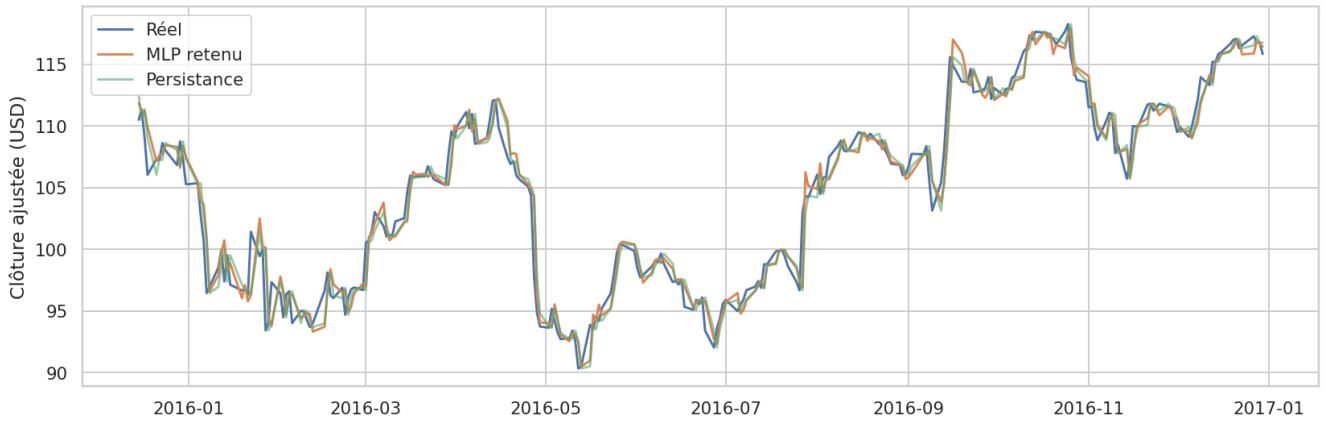


Figure 5 — Clôtures réelles, MLP retenu et persistance sur les 264 observations de test.

6.2 Comprendre les métriques

MAE = moyenne des erreurs absolues : elle donne une erreur typique en dollars. **RMSE** = racine de la moyenne des erreurs au carré : elle pénalise davantage les grosses erreurs. **R²** compare l'erreur du modèle à celle d'une prédiction constante égale à la moyenne des valeurs évaluées. Il peut être négatif et ne mesure pas la rentabilité.

Une accuracy classique n'est pas calculée : demander si deux prix continus sont exactement égaux n'est pas une mesure pertinente. MSE/époques et RMSE/époques remplacent donc la paire loss/accuracy demandée dans le sujet, avec justification explicite.

7. Classification : comprendre les données

Le fichier predictive_maintenance.csv contient 10 000 observations et dix colonnes. La cible Failure Type distingue six situations : No Failure, Heat Dissipation Failure, Power Failure, Overstrain Failure, Tool Wear Failure et Random Failures. Il s'agit d'un jeu synthétique, pas d'une validation sur un parc réel de machines.

Colonne	Utilisation
Air temperature [K]	Température de l’air : variable numérique
Process temperature [K]	Température du procédé : variable numérique
Rotational speed [rpm]	Vitesse de rotation : variable numérique
Torque [Nm]	Couple : variable numérique
Tool wear [min]	Usure de l’outil : variable numérique
Type	Catégorie de produit L, M ou H : variable catégorielle
Failure Type	Cible multiclasse à prédire
Target	Exclue : indique déjà la présence d’une panne
UDI, Product ID	Exclus : identifiants, pas des mesures physiques

7.1 Nettoyage et contrôles

Les contrôles du fichier brut trouvent 0 valeur manquante et 0 doublon exact. Les pipelines conservent néanmoins une imputation pour rendre le traitement explicite et robuste. Les valeurs numériques non finies sont traitées comme manquantes. Les observations sans étiquette seraient retirées.

Le jeu présente 27 désaccords entre l'indicateur Target et la présence d'un type de panne. Ces désaccords sont signalés et les étiquettes Failure Type sont conservées, sans correction arbitraire. Les valeurs extrêmes ne sont pas supprimées automatiquement : elles peuvent caractériser une panne réelle.

7.2 Partage stratifié

Les partitions comprennent 7 000 observations de train, 1 500 de validation et 1 500 de test. Le tirage est stratifié sur Failure Type et la graine vaut 42. Chaque partition conserve les six classes et leurs proportions approximatives. Les identifiants UDI sont disjoints entre les partitions.

8. Classification : EDA et rééquilibrage

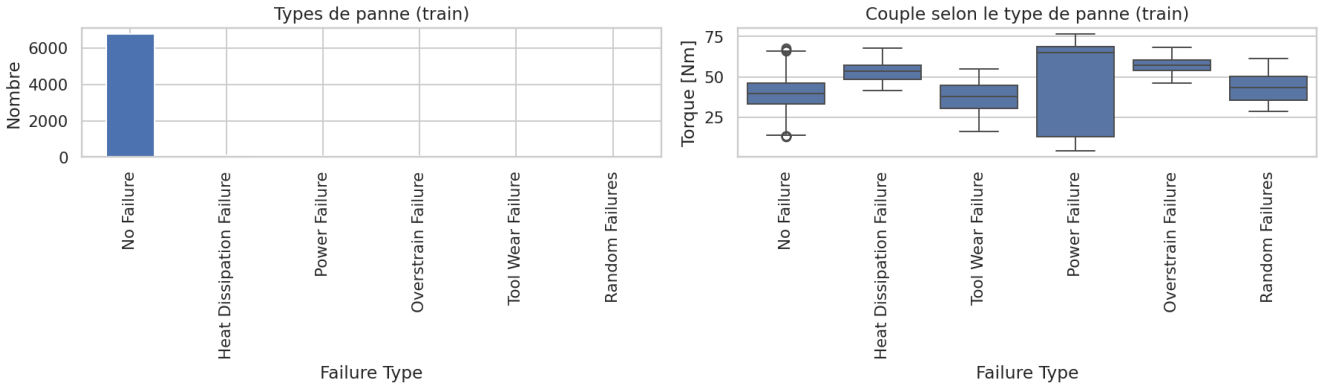


Figure 6 — Déséquilibre des classes et distribution du couple par type de panne sur le train.

La classe No Failure domine largement. Dans le train, elle compte 6 756 cas, contre seulement 13 pannes aléatoires. Un modèle qui minimise l'erreur globale sans précaution peut donc privilégier l'absence de panne. L'EDA des mesures et des corrélations aide aussi à repérer les échelles et les relations, sans supprimer automatiquement les cas rares.

Classe du train	Avant SMOTENC	Après SMOTENC
Heat Dissipation Failure	78	6756
No Failure	6756	6756
Overstrain Failure	55	6756
Power Failure	67	6756
Random Failures	13	6756
Tool Wear Failure	31	6756

8.1 Pourquoi SMOTENC et dans quel ordre ?

Les mesures sont imputées par leur médiane et standardisées. Le type est imputé par son mode, puis encodé temporairement par 0, 1 ou 2. SMOTENC est informé que la sixième colonne est catégorielle : il interpole les mesures tout en conservant une catégorie valide. Ensuite seulement, le type est encodé en trois indicateurs one-hot.

Cette méthode évite de créer par interpolation un type de produit fictif tel que « moitié L, moitié M ». Les six classes du train final sont portées à 6 756 observations chacune, soit 40 536 lignes. Ces exemples synthétiques ne sont pas des mesures physiques supplémentaires et ne garantissent pas une meilleure généralisation.

Le nombre de voisins de SMOTENC est fixé à trois pour rester compatible avec les faibles effectifs minoritaires dans les folds. La standardisation des mesures suit z = (x - moyenne_train) / écart_type_train. Elle évite que l'échelle d'une mesure domine le calcul des distances entre voisins.

Dans GridSearchCV, le même pipeline est réajusté dans chaque fold : l'imputation, la standardisation et SMOTENC ne voient pas les données du fold de validation. Ni la validation indépendante ni le test ne sont rééquilibrés.

8.2 Lire les distributions et les corrélations

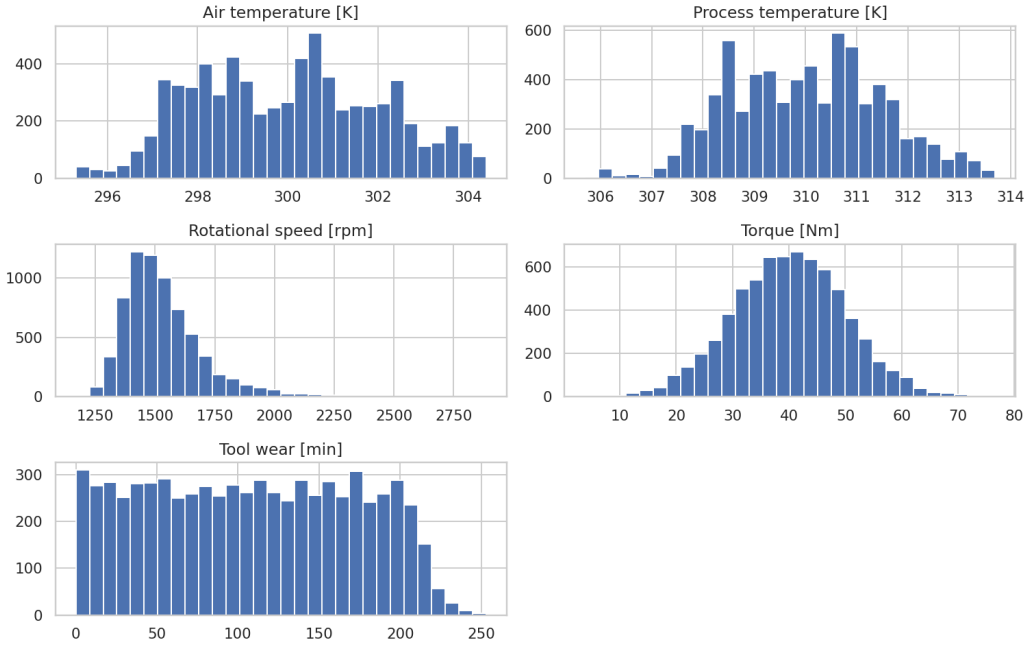


Figure 7 — Distributions des cinq mesures numériques sur le train.

Ces histogrammes permettent de comparer les unités, les étendues et la forme des distributions. Une mesure n'a pas besoin de suivre une loi normale pour être standardisée. Une valeur éloignée du centre n'est pas automatiquement une erreur : la supprimer pourrait retirer précisément des situations de panne.

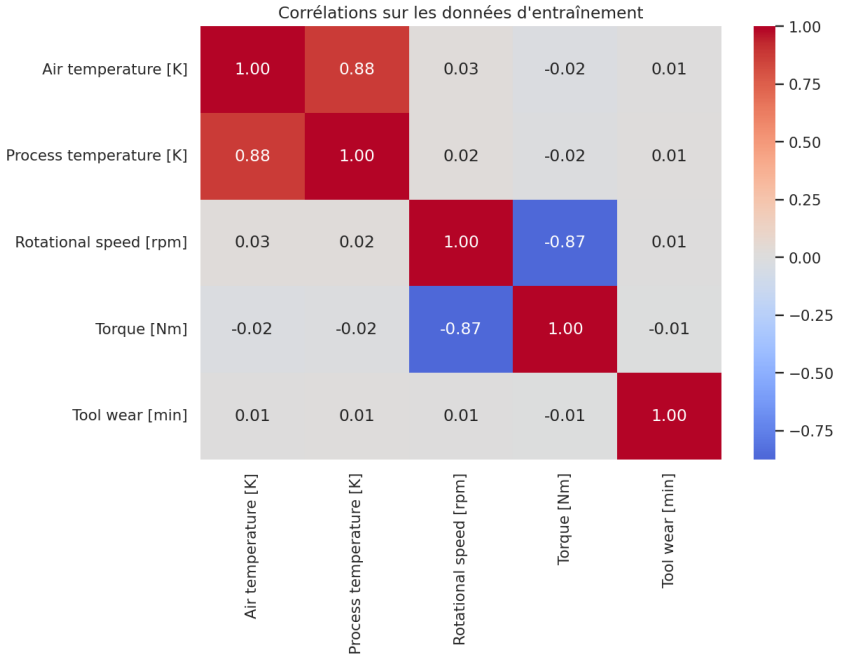


Figure 8 — Corrélations linéaires entre les cinq mesures du train.

La température de l'air et celle du procédé sont liées ; vitesse et couple peuvent aussi présenter une relation. Une corrélation décrit une association linéaire, pas une causalité. Elle ne suffit pas à prévoir le type de panne et ne remplace pas l'évaluation du réseau. Les graphiques de rééquilibrage complémentaires sont conservés dans results/classification/ et le notebook.

9. Classification : MLP et recherche

9.1 Architecture retenue

```
5 mesures + 3 indicateurs de Type = 8 entrées
Linear(8, 64) → ReLU → Dropout(0.2)
Linear(64, 32) → ReLU → Dropout(0.2)
Linear(32, 6) → six logits
```

Le réseau possède 2 854 paramètres entraînables. Un logit est un score brut associé à une classe. Pendant l'apprentissage, CrossEntropyLoss reçoit directement ces logits : ajouter un softmax dans le modèle avant cette loss serait inutile et pourrait nuire au calcul numérique. Pour obtenir des probabilités à la prédiction, predict_proba applique softmax ; predict choisit le plus grand logit.

9.2 Critère de recherche

La grille comporte les mêmes neuf configurations que la régression et utilise StratifiedKFold avec trois folds mélangés, graine 42. Le score maximisé est le F1 macro, qui donne autant de poids à chaque classe. La meilleure configuration est [64, 32], Adam, taux 0,01 et 80 époques. Son F1 macro CV moyen vaut 0.5579.

Le F1 macro CV du modèle initial est distinct du F1 macro de la validation indépendante et de celui du test. Ils sont calculés sur des observations et des modèles ajustés différents ; ils n'ont donc aucune raison d'être identiques.

9.3 Régularisation comparée

Variante	Réglage
initial	Sans Dropout, sans pénalité L2, budget complet
dropout	Probabilité de désactivation = 0,2 après chaque couche cachée
l2	weight_decay = 0,001 dans l’optimiseur
early_stopping	Patience = 10 ; restauration des poids à la meilleure loss validation
combined	Dropout + L2 + arrêt anticipé

Le Dropout désactive aléatoirement des neurones pendant le train, mais est désactivé par model.eval() pour les mesures. L2 pénalise les poids importants. L'arrêt anticipé limite la durée d'apprentissage. Pour isoler leurs effets, les variantes gardent la même architecture, la même graine et le même budget maximal.

10. Classification : résultats globaux

Variante	F1 macro validation	Accuracy test	Balanced acc. test	F1 macro test
initial	0.5000	0.9593	0.6335	0.5539
dropout	0.5208	0.9333	0.6799	0.5668
l2	0.4706	0.9120	0.7238	0.5092
early_stopping	0.4733	0.8993	0.6883	0.5394
combined	0.4538	0.8953	0.7347	0.4861

La variante **dropout** est retenue sur la validation. Elle obtient un F1 macro validation de 0,5208, contre 0,5000 pour le modèle initial. La combinaison de toutes les régularisations ne donne pas le meilleur résultat : une contrainte trop forte peut empêcher le réseau de bien apprendre les classes.

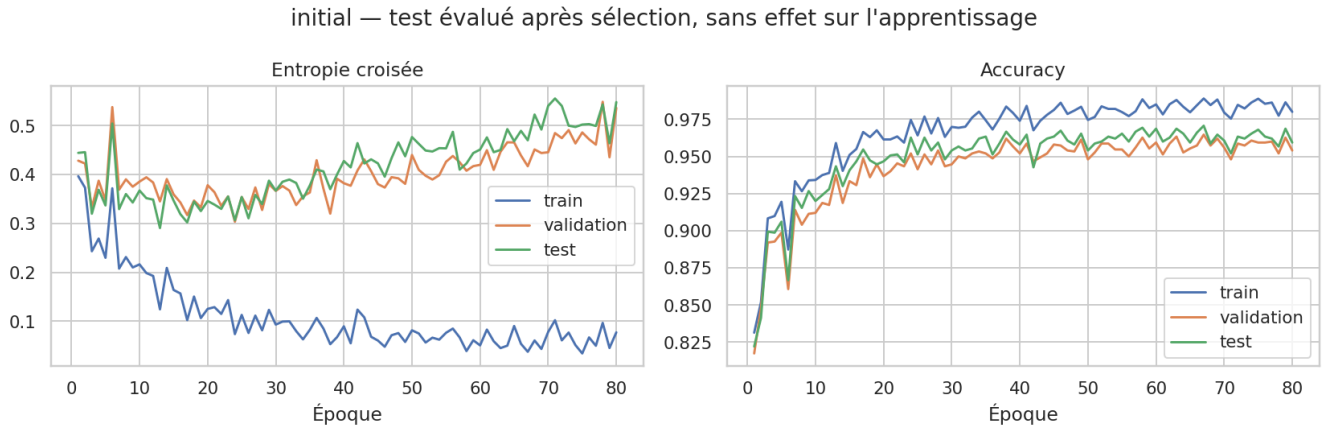


Figure 9 — Entropie croisée et accuracy du modèle initial.

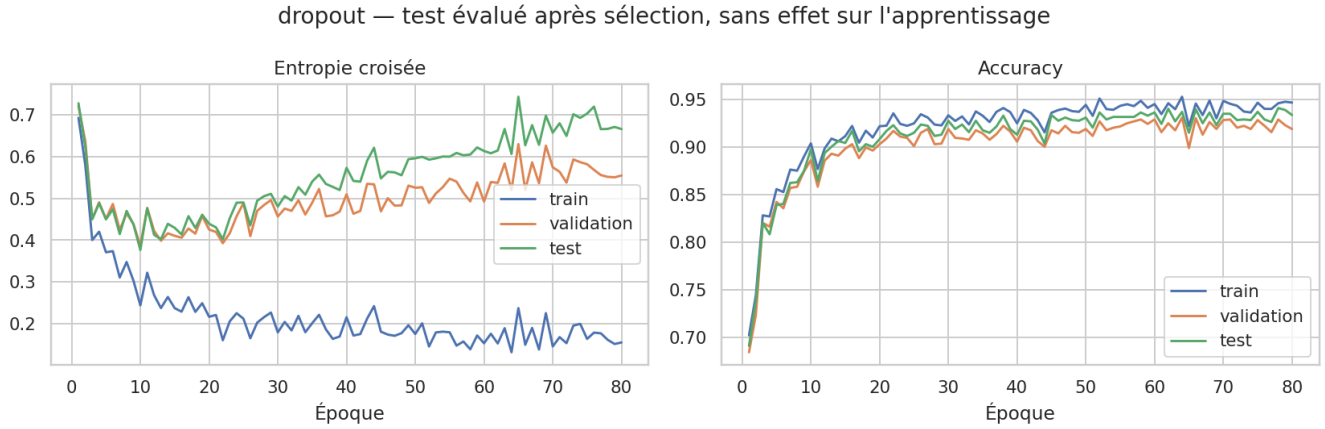


Figure 10 — Entropie croisée et accuracy avec Dropout, modèle retenu.

Le modèle initial atteint un F1 macro train de 0,7959, contre 0,5000 sur validation : cet écart est compatible avec une généralisation limitée. Avec Dropout, les scores sont 0,6466 et 0,5208. Le train baisse et la validation progresse légèrement : la régularisation réduit ici l'écart, sans résoudre toutes les difficultés.

10.1 Pourquoi le score de référence est trompeur

Prédire toujours No Failure donne 96.53% d'accuracy test, mais seulement 0.1637 de F1 macro et 16,67 % de balanced accuracy. Le réseau retenu donne 93.33% d'accuracy et 0.5668 de F1 macro. Il fait davantage d'erreurs globales, mais reconnaît une partie des classes de panne que la référence ignore entièrement.

Il n'est donc pas correct de conclure que le réseau est moins utile seulement parce que son accuracy est inférieure à celle de la classe majoritaire. Inversement, son F1 macro supérieur ne suffit pas à garantir son utilité industrielle : les coûts des fausses alertes et des pannes manquées ne sont pas modélisés.

11. Classification : métriques par classe

Pour une classe donnée, la **précision** répond à : « Parmi les alertes de cette classe, combien sont justes ? » Le **rappel**, ou sensibilité, répond à : « Parmi les vrais cas de cette classe, combien sont détectés ? » Le **F1** est la moyenne harmonique de précision et rappel. Le **support** est le nombre réel de cas de cette classe.

L'accuracy est la proportion totale de prédictions correctes. La balanced accuracy est la moyenne des rappels des classes. Le F1 macro moyenne leurs F1 à poids égaux ; le F1 pondéré les moyenne selon leurs effectifs, et peut donc rester élevé même si des classes rares sont ignorées.

Pour une classe, précision = TP / (TP + FP), rappel = TP / (TP + FN), et F1 = 2 × précision × rappel / (précision + rappel). TP signifie vrai positif, FP faux positif et FN faux négatif. Quand une métrique est indéfinie faute de prédictions ou de cas, le rapport utilise zero_division=0.

11.1 Modèle retenu — entraînement non augmenté

Classe	Précision	Rappel	F1	Support
Heat Dissipation Failure	0.7333	0.9872	0.8415	78
No Failure	0.9997	0.9448	0.9715	6756
Overstrain Failure	0.7534	1.0000	0.8594	55
Power Failure	0.6147	1.0000	0.7614	67
Random Failures	0.1121	0.9231	0.2000	13
Tool Wear Failure	0.1403	1.0000	0.2460	31

11.2 Modèle retenu — test naturel

Classe	Précision	Rappel	F1	Support
Heat Dissipation Failure	0.6296	1.0000	0.7727	17
No Failure	0.9949	0.9365	0.9648	1448
Overstrain Failure	0.7059	1.0000	0.8276	12
Power Failure	0.6667	1.0000	0.8000	14
Random Failures	0.0000	0.0000	0.0000	2
Tool Wear Failure	0.0204	0.1429	0.0357	7

Sur le test, les rappels des pannes thermiques, de puissance et de surcharge atteignent 100 %, mais sur 17, 14 et 12 observations seulement. Pour ces classes, la précision est inférieure à 100 %, donc des fausses alertes existent. Un rappel parfait sur un petit échantillon n'est pas une garantie pour des données futures.

L'usure de l'outil reste mal reconnue : un seul cas sur sept est détecté, avec une précision très faible. Les deux pannes aléatoires ne sont pas reconnues. Le déséquilibre initial, le faible nombre d'exemples originaux et le caractère aléatoire de certaines étiquettes limitent ce que l'augmentation peut apporter.

12. Matrice de confusion et diagnostic

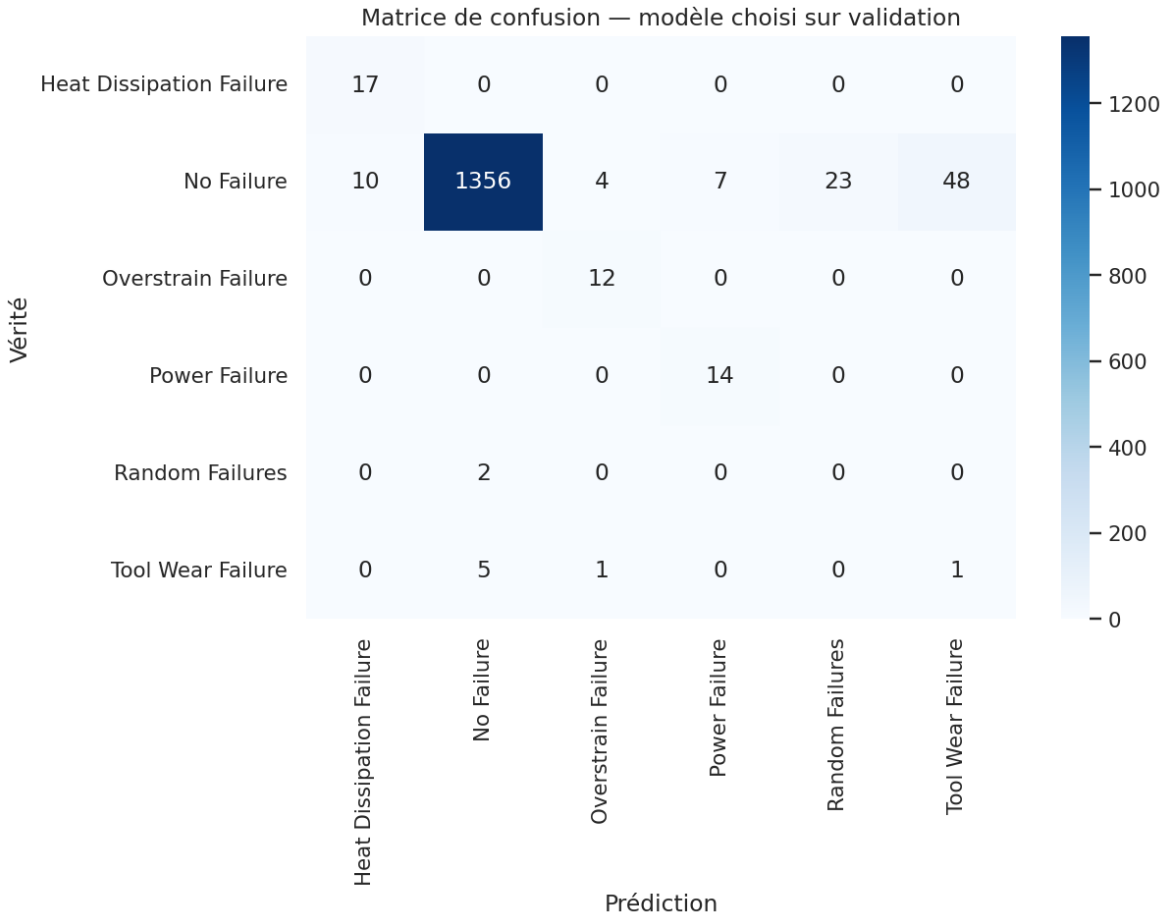


Figure 11 — Modèle Dropout retenu : lignes = classes réelles ; colonnes = classes prédites.

Les valeurs de la diagonale représentent les prédictions correctes. Les valeurs hors diagonale indiquent les confusions : par exemple, une machine sans panne prédite en usure d'outil correspond à une fausse alerte. Une panne réelle prédite No Failure correspond au contraire à une panne manquée.

Dans ce test, 92 observations sans panne sont classées comme une panne, et 7 observations présentant une panne sont classées No Failure. Ce regroupement binaire sert seulement à lire les erreurs ; le modèle reste bien multiclasse.

12.1 Limites et améliorations possibles

Les résultats portent sur une seule graine, une seule partition de test et un jeu synthétique. Aucun intervalle de confiance ni test de significativité n'est calculé. Les scores par classe rare ont une forte incertitude ; les deux seules pannes aléatoires du test ne permettent pas une estimation stable.

Pour aller plus loin, on pourrait comparer plusieurs partitions, une pondération de la loss, d'autres stratégies de rééquilibrage ou des variables physiques dérivées. Ces pistes sont proposées, pas exécutées dans ce rapport. Si elles servent à choisir un nouveau modèle après lecture du test actuel, il faudra prévoir un nouveau test indépendant pour l'évaluation finale.

13. Organisation du code et reproduction

Fichier ou dossier	Responsabilité
src/data.py	Sources figées, SHA-256, téléchargement, variables et partitions
src/models.py	MLP, fit/predict, standardisation cible, états par époque
src/experiments.py	EDA, pipelines, GridSearchCV, variantes et résultats
notebooks/	Deux parcours pédagogiques commentés avec sorties enregistrées
results/	Scores, historiques, figures, rapports et métadonnées
tests/test_workflow.py	Quatre contrôles de données et de pipelines réels
scripts/setup.sh	Environnement virtuel et dépendances épinglées
scripts/generate_report.py	Génération du présent PDF sans réentraîner

13.1 Installation et entraînement

```
git clone https://github.com/KarmasHammam/Atelier1_DeepLearning.git
cd Atelier1_DeepLearning
bash scripts/setup.sh
source .venv/bin/activate
python -m src.experiments --task all
python -m unittest discover -s tests -v
python scripts/write_readme.py
python scripts/execute_notebooks.py
```

L'installation CPU ne demande ni GPU ni clé Kaggle. Les fichiers bruts sont téléchargés dans data/raw/ et exclus de Git. Les scripts ne modifient pas le Python système. Le mode --quick réduit le budget à 8/16 époques ; il doit être écrit dans results_quick pour ne pas remplacer les résultats complets du rapport.

13.2 Générer ce document

```
python -m pip install -r requirements-report.txt
python scripts/generate_report.py
```

Le générateur lit les CSV/JSON et figures existants. Il ne relance aucun apprentissage. Les dépendances de rapport sont séparées de celles nécessaires aux expériences.

13.3 Vérifications réalisées

Les deux recherches complètes et les dix modèles ont terminé. Les quatre tests automatiques passent : cible temporelle et frontières purgées ; partitions de classification et catégories valides après augmentation ; clonage du régresseur et standardisation de sa cible ; absence de modification du modèle lors de l'évaluation rétrospective du test. Les deux notebooks ont également été exécutés sans erreur, avec huit cellules de code chacun. Les tests ne prouvent pas une performance industrielle ; ils contrôlent des risques précis du workflow.

14. Synthèse personnelle et conclusion

Cet atelier m'a permis de mieux comprendre comment construire et évaluer un réseau de neurones avec PyTorch. J'ai appris à distinguer la régression, qui prédit une valeur numérique, de la classification multiclasse, qui prédit une catégorie. J'ai aussi mieux compris les étapes de l'apprentissage : calculer les prédictions, mesurer l'erreur, calculer les gradients et mettre à jour les poids.

J'ai compris que la préparation des données est aussi importante que le choix du réseau. Pour les cours boursiers, il faut respecter l'ordre chronologique pour ne pas utiliser le futur pendant l'apprentissage. Pour les pannes, il faut tenir compte du déséquilibre des classes et appliquer le rééquilibrage uniquement sur les données d'entraînement.

Les résultats m'ont montré qu'un bon score ne suffit pas à conclure qu'un modèle est performant. En régression, le réseau fait presque aussi bien que la méthode simple qui reprend le dernier cours connu. En classification, une accuracy élevée peut masquer une mauvaise détection des pannes rares. C'est pour cela que je regarde aussi le F1 macro, le rappel par classe et la matrice de confusion.

Enfin, la comparaison des modèles m'a permis de comprendre que le Dropout, la régularisation L2 et l'arrêt anticipé n'améliorent pas automatiquement les résultats. Il faut vérifier leur effet sur la validation et garder le test pour l'évaluation finale. Ce que je retiens surtout de cet atelier, c'est qu'il faut expliquer les résultats et leurs limites, au lieu de chercher seulement le meilleur score.

Conclusion générale

Les deux tâches demandées ont été réalisées avec des MLP PyTorch, une recherche GridSearchCV, des courbes d'apprentissage et plusieurs régularisations. Le protocole conserve une distinction nette entre entraînement, validation et test. Les notebooks et le code permettent de retrouver chaque étape.

Le réseau boursier ne dépasse que très faiblement la persistance sur le test et reste moins bon qu'elle sur validation. Le réseau de classification améliore la prise en compte de plusieurs classes, mais échoue encore sur certaines pannes rares. Ces limites font partie du résultat : une expérimentation sérieuse ne consiste pas à présenter seulement des scores élevés, mais à expliquer ce qu'ils mesurent et ce qu'ils ne démontrent pas.

Annexe A — Détail de la recherche CV

Les tableaux ci-dessous reprennent les neuf configurations réellement évaluées. La valeur ± indique l'écart-type des scores sur les trois folds ; ce n'est pas un intervalle de confiance. Les architectures sont les tailles des couches cachées. Les scores de régression sont remis en RMSE positive pour faciliter la lecture.

A.1 Régression : RMSE moyenne CV

Rang	Couches	Optim.	Taux	Époques	Moyenne ± écart-type
1	(32,)	adam	0.01	80	1.3680 ± 0.0962
2	(32,)	adam	0.01	40	1.5142 ± 0.1543
3	(64, 32)	adam	0.01	80	1.6442 ± 0.3590
4	(64, 32)	adam	0.01	40	1.6917 ± 0.3855
5	(64, 32)	adam	0.001	80	1.9303 ± 0.6737
6	(64, 32)	sgd	0.01	80	2.4683 ± 1.1514
7	(64, 32)	adam	0.001	40	3.5669 ± 2.1699
8	(32,)	adam	0.001	80	4.2324 ± 2.4273
9	(32,)	adam	0.001	40	10.4599 ± 5.4206

A.2 Classification : F1 macro moyen CV

Rang	Couches	Optim.	Taux	Époques	Moyenne ± écart-type
1	(64, 32)	adam	0.01	80	0.5579 ± 0.0181
2	(64, 32)	adam	0.001	80	0.5507 ± 0.0155
3	(32,)	adam	0.01	80	0.5458 ± 0.0073
4	(64, 32)	adam	0.01	40	0.5354 ± 0.0149
5	(32,)	adam	0.01	40	0.5258 ± 0.0079
6	(64, 32)	sgd	0.01	80	0.5207 ± 0.0035
7	(64, 32)	adam	0.001	40	0.5098 ± 0.0048
8	(32,)	adam	0.001	80	0.4807 ± 0.0177
9	(32,)	adam	0.001	40	0.4378 ± 0.0033

La recherche reste volontairement limitée. Elle explore surtout Adam et une seule configuration SGD. Elle ne permet donc pas de conclure qu'Adam est toujours supérieur à SGD, ni que l'architecture retenue est un optimum global.

Annexe B — Lire les points clés du code

B.1 Créer une cible future sans la mettre dans les entrées

```
stock['return_1'] = stock.close.pct_change(fill_method=None)
stock['mean_5'] = stock.close.rolling(5).mean()
stock['std_5'] = stock.close.rolling(5).std()
stock['next_close'] = stock.close.shift(-1)
stock['target_date'] = stock.date.shift(-1)
```

rolling(5) regarde la séance courante et les quatre précédentes. shift(-1) est réservé à la cible et à sa date. REG_FEATURES n'inclut jamais next_close ni target_date.

B.2 Encapsuler les transformations pendant la CV

```
search = GridSearchCV(
    make_pipeline(task), parameter_grid(quick=False),
    cv=cv, scoring=scoring, n_jobs=1,
    refit=False, error_score='raise'
)
search.fit(X_train, y_train)
```

Le pipeline empêche d'ajuster StandardScaler ou SMOTENC avant la séparation en folds. refit=False signifie qu'aucun modèle final automatique n'est conservé : les variantes sont ensuite entraînées explicitement avec les paramètres sélectionnés.

B.3 Mesurer sans poursuivre l’apprentissage

```
model.eval()
with torch.no_grad():
    logits = model(features)
```

eval() désactive notamment le Dropout. no_grad() évite de construire le graphe de calcul des gradients. Les états de chaque époque sont conservés pendant les expériences finales ; les courbes de test sont obtenues après choix de la variante, puis les poids du modèle retenu sont restaurés. Aucun score de test ne pilote les gradients, la patience ou la sélection.

B.4 Arrêt anticipé : époques exécutées et retenues

Tâche	Variante	Époques exécutées	Époque des poids retenus
Régression	early_stopping	32	22
Régression	combined	32	22
Classification	early_stopping	13	3
Classification	combined	32	22

Les dernières époques peuvent ne pas améliorer la validation. Après dix époques sans amélioration suffisante, l'entraînement s'arrête et recharge les poids de la meilleure loss de validation, pas nécessairement ceux de la dernière époque exécutée.

Annexe C — Sources, traçabilité et glossaire

Dépôt du projet : github.com/KarmasHammam/Atelier1_DeepLearning. Les notebooks, métriques détaillées et scripts complètent ce rapport.

Les archives Kaggle n'étaient pas accessibles depuis l'environnement utilisé. Des copies publiques attribuées aux jeux du sujet ont donc été utilisées. Leurs URL sont figées sur des commits. Le contrôle SHA-256 garantit la stabilité de ces copies, mais leur identité octet par octet avec les archives Kaggle n'a pas été vérifiée. Les données brutes ne sont pas redistribuées dans le dépôt.

prices-split-adjusted.csv

Source du sujet : https://www.kaggle.com/datasets/dgawlik/nyse

Copie utilisée : téléchargement du fichier figé sur GitHub.

```
SHA-256 :
0a8e0505591c8ae8c545d6d9e7bb6973
5c18568745ef848bf4d01e1afd871ec5
```

predictive_maintenance.csv

Source du sujet :
https://www.kaggle.com/datasets/shivamb/machine-predictive-maintenance-classification

Copie utilisée : téléchargement du fichier figé sur GitHub.

```
SHA-256 :
ba1eb264c69e69258acb2df6991055af
c93092b1be436a29d2e6037b0ebd80cc
```

Documentation des outils

PyTorch : modules et apprentissage

scikit-learn : GridSearchCV

imbalanced-learn : SMOTENC

Petit glossaire

Terme	Sens dans le projet
DNN / MLP	Réseau profond / perceptron multicouche à couches entièrement connectées
Loss	Fonction d’erreur utilisée pour apprendre les poids
Hyperparamètre	Réglage choisi avant l’apprentissage, par exemple le taux ou l’architecture
Fold	Sous-partition utilisée pendant la validation croisée
Surapprentissage	Meilleure adaptation au train qu’à de nouvelles observations
Logit	Score brut avant transformation en probabilité de classe
Fuite de données	Information du futur ou de l’évaluation utilisée pour entraîner ou choisir